

Bandit - Level 13

Goal

The password for the next level is stored in `/etc/bandit_pass/bandit14`, which can only be read by user **bandit14**. There is no password to retrieve directly — the home directory contains an SSH private key instead.

Connection

```
ssh -p 2220 bandit13@bandit.labs.overthewire.org
```

Use the password retrieved at the end of [Bandit - Level 12](#).

Concepts

This level breaks with everything from 8 to 12. No text processing, no file formats — the subject is **authentication**.

Key-based authentication

SSH accepts two ways of proving identity:

Method	What you prove	What the server stores
Password	you <i>know</i> a secret	a hash of the password
Public key	you <i>possess</i> a private key	the matching public key

With key-based auth the private key **never travels**. The server sends a challenge, the client signs it with the private key, and the server verifies that signature against the public key it holds. A network observer sees only the challenge and the signature — neither of which reveals the key.

```
ssh -i <keyfile> -p 2220 user@host
```

`-i` stands for **identity file**.

Where `-i` looks for the key

Nowhere special. `-i` resolves the path like any other command: relative to the current working directory unless given an absolute path.

```
ssh -i sshkey.private ...      # relative - depends on where you are
ssh -i .\sshkey.private ...    # same thing, made explicit
ssh -i C:\Users\David\sshkey.private ...  # absolute - works from anywhere
```

The command in this level only worked because `cd $env:USERPROFILE` had been run first. Launched from a different folder with the relative form, it fails — the same working-directory assumption that caused trouble in [Bandit - Level 07](#) and [Bandit - Level 08](#). `pwd` first, as always.

The default location, when `-i` is omitted

Without `-i`, SSH looks in `~/.ssh/` (on Windows, `C:\Users\<user>\.ssh\`) for a fixed set of filenames:

```
~/.ssh/id_ed25519
~/.ssh/id_ecdsa
~/.ssh/id_rsa
```

This is why anyone administering servers rarely types `-i`: the key is saved under one of those names and `ssh host` just works. `-i` exists for the opposite case — a key that isn't where SSH would look, or several keys to choose between.

That directory also appeared as an error earlier in this level:

```
Could not create directory '/home/bandit13/.ssh' (Permission denied).
```

SSH was trying to create it to store `known_hosts` — the record of previously visited servers and their host fingerprints — and couldn't, because the Bandit home is read-only. On the local Windows machine that directory does exist, and it's where the server's fingerprint was saved after answering `yes` to *"The authenticity of host can't be established"*.

Tidier organisation once keys start accumulating:

```
mkdir $env:USERPROFILE\.ssh -Force
move sshkey.private $env:USERPROFILE\.ssh\bandit14.key
ssh -i $env:USERPROFILE\.ssh\bandit14.key -p 2220
bandit14@bandit.labs.overthewire.org
```

An absolute path works from any directory. SSH also supports a `~/.ssh/config` file where a host alias, user, port and key can be declared once and invoked as `ssh bandit14` — worth setting up after Level 15, alongside the SSH concept note.

`<...>` In documentation is a placeholder, never literal syntax. Typing `ssh -i <sshkey.private>` makes bash read `<` as input redirection and `>` as output redirection — it tried to *create a file named* `-p`, hence `-bash: -p: Permission denied`. See [Linux - Piping and Redirection](#).

Why this matters

A stolen private key is worse than a stolen password: **there is no password to change**. Whoever holds it *is* that user until someone revokes the public key on every server that trusts it.

That's why `~/.ssh/id_rsa` and `id_ed25519` are among the first files an attacker looks for after compromising a host, and why lateral movement across a fleet so often runs on harvested keys. Kasui Domain 2 covers this under access control; Domain 4 under identity management.

Permissions as the whole point

Everything in this level is decided by permissions. `ls -la` in `/etc/bandit_pass`:

```
-r----- 1 bandit14 bandit14    33 bandit14
| | |      |
| | |      |
| | |      | others: nothing
| | |      | group: nothing
| | |      | owner: read only
| | |      | regular file, NOT a directory
```

Only `bandit14` can read it. The file is visible and unreadable — exactly the intended design.

And in the home directory:

```
-rw-r----- 1 bandit14 bandit13 2602 sshkey.private
```

└── group is bandit13 → group bits r-- apply to you

Owned by `bandit14` , grouped to `bandit13` . Placed there deliberately so this level can be solved.

Attempts

```
cd /etc/bandit_pass/bandit14
# → -bash: cd: /etc/bandit_pass/bandit14: Not a directory
# Repeated five times from different working directories, changing the path
# each time. The path was never the problem: `bandit14` is a FILE. `cd` only
# works on directories. The error said "Not a directory", not "No such file".

wc -c /etc/bandit_pass/bandit14
# → Permission denied
# Correct diagnosis: the file exists, is visible, and is unreadable by bandit13.
```

Then, back in the home directory:

```
ls -la
# → HINT  sshkey.private

cat sshkey.private
# → -----BEGIN OPENSSSH PRIVATE KEY----- ...

wc -c sshkey.private && wc -c HINT
# → 2602 sshkey.private
#    467 HINT
```

That 2602 turned out to matter later. Recorded almost as an afterthought, it became the fact that solved the level.

The four errors, in order

```
ssh -i <sshkey.private> -p 2220 bandit14@bandit.labs.overthewire.org
# → -bash: -p: Permission denied
# The angle brackets were typed literally and bash read them as redirections.
```

```
ssh -i sshkey.private -p 2220 bandit14@bandit.labs.overthewire.org
# → !!! You are trying to log into this SSH server from localhost.
#   !!! Connecting from localhost is blocked to conserve resources.
```

Note the resolved address in the same output: `[bandit.labs.overthewire.org]:2220` (`[127.0.0.1]:2220`) . From inside the Bandit server, the public hostname resolves to itself. Writing the public name doesn't change that it's localhost. This was point 3 of the HINT, already read and not applied.

After copying the key to Windows and connecting from there:

```
WARNING: UNPROTECTED PRIVATE KEY FILE!
Permissions for 'sshkey.private' are too open.
Bad permissions. Try removing permissions for user: DESKTOP-...\Gaming
```

SSH refuses to use a key other users can read, rather than letting a compromised credential be used silently. On Windows the culprit was an inherited ACL from the parent folder.

```
Load key "sshkey.private": invalid format
```

Two separate causes, one after the other: Windows CRLF line endings, and then a missing trailing newline.

Solution

1. Read the key on the server and copy it:

```
cat sshkey.private      # select from -----BEGIN----- to -----END----- inclusive
exit                   # localhost is blocked – the connection must come from
outside
```

2. Save it locally (Notepad, or any editor). Watch for a `.txt` extension being appended silently.

3. Fix line endings and the trailing newline (PowerShell):

```
icacls sshkey.private /grant "$($env:USERNAME):(F)" # write access first

$k = (Get-Content sshkey.private -Raw) -replace "`r`n", "`n"
if (-not $k.EndsWith("`n")) { $k += "`n" }
[System.IO.File]::WriteAllText("$env:USERPROFILE\sshkey.private", $k)

(Get-Content sshkey.private -Raw).Length # → 2602, matching the server
```

`[System.IO.File]::WriteAllText` writes exactly what it's given. `Out-File` and `>` would reinsert CRLF — the very problem being fixed.

4. Restrict permissions — after editing, never before:

```
icacls sshkey.private /inheritance:r
icacls sshkey.private /grant:r "$($env:USERNAME):(R)"
```

```
icacls sshkey.private # → DESKTOP-...\David:(R) and nobody else
```

`/inheritance:r` severs inheritance from the parent folder and drops inherited entries. In `/grant:r`, the `:r` means **replace**, not read.

5. Connect:

```
ssh -i sshkey.private -p 2220 bandit14@bandit.labs.overthewire.org
```

Linux equivalent for when the Slimbook is back:

```
chmod 600 sshkey.private # owner rw, group and others nothing
```

Same requirement, two permission models — see [Linux - Permissions & Process Management](#).

Points of Friction

1. Five attempts at `cd` on a file. The error message said `Not a directory` every time, and the fix was to read it rather than vary the path. `cd` operates on directories; the leading `-` in `ls -l` output distinguishes a regular file from a `d` directory.

2. Typed `<...>` literally. Angle brackets in documentation mark a placeholder. In a shell they are redirection operators, which is why the error mentioned a permission problem with `-p` — bash was trying to create a file with that name.

3. Ran `ssh` from inside the server. The HINT stated the localhost restriction explicitly, and the connection output showed the hostname resolving to `127.0.0.1`. Both were read; neither was applied. Point 4 of that same HINT — *"If you get errors, read the error message on your screen. We mean it!"* — was the whole lesson of the level.

4. Restricted permissions before finishing the edit. `icacls /grant:r "David:(R)"` granted read-only, then `WriteAllText` failed with *Access to the path is denied* — locked out of a file by a permission set applied one step too early.

Small version of a real operational failure: applying a restrictive policy before the configuration is complete, and losing the access needed to complete it. On a firewall or a remote host, that ends the session permanently.

5. Chased the format error before measuring it. CRLF was removed, the error persisted, and two more fixes were attempted blind. Comparing sizes settled it in one command: **2602 on the server, 2601 locally**. One byte — the trailing newline.

Key Takeaway

When something should be identical and isn't working, compare sizes before inspecting contents. `wc -c` at the source, `.Length` at the destination. A one-byte difference points straight at the cause without reading 2,600 characters by eye.

The same counting reflex already appeared twice: `wc -l` mapping the file structure in [Bandit - Level 08](#), and `ls -la` revealing the zeroed file in [Bandit - Level 12](#). Counting is cheap and it doesn't lie.

The second takeaway is about **SSH private keys**. Three separate guards had to be satisfied before the key would even be considered: correct file permissions, Unix line endings, and a trailing newline. None of them are arbitrary — SSH refuses a key it can't trust rather than using it and hoping, because **a leaked private key has no password to rotate**.

Next

```
ssh -i sshkey.private -p 2220 bandit14@bandit.labs.overthewire.org
```

◀ Previous: [Bandit - Level 12](#) · Next ▶ [Bandit - Level 14](#)